

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA DE MINAS GERAIS
Engenharia da Computação
Compiladores

Analisador Semântico

Alunos: Gabriel Bernalle Barbosa Matozinhos

Professora: Kecia Marques

Belo Horizonte, junho de 2025

Objetivo

A terceira etapa do trabalho prático teve como objetivo implementar o analisador semântico da linguagem definida, com base na gramática fornecida na etapa anterior. Também foi necessário criar e gerenciar uma tabela de símbolos para garantir a verificação correta de declarações e usos de variáveis, além da compatibilidade entre tipos em expressões e comandos.

Processo

Diferente das outras etapas em que expliquei o que foi criado, o que faz e correções; para essa parte do trabalho irei separar em dois tópicos, um explicando o que foi criado e outro para o que foi alterado, para assim ficar mais fácil a compreensão dos processos e visualização do correto funcionamento.

Criado

- A primeira coisa que foi feita, foi a criação de uma pasta nova chamada “Semantico”, que nela somente conterá a Tabela de Símbolos feita para o Analisador Semântico.

Esse arquivo, “SymbolTable.java”, possui três métodos, um responsável pela declaração de tupla (Chave, valor), outra para verificar se uma variável já foi declarada dentro da Tabela de Símbolos, e a última para obter o tipo de uma variável na Tabela de Símbolos. Importante destacar que aqui foi feito a tabela utilizando HashMap, pois fica mais robusta e elegante a aplicação dessa forma.

- Todas as demais alterações foram feitas na pasta do Analisador Sintático, arquivo “SyntaticAnalysis.java”. Foi feita a criação de uma função auxiliar criada para inferir o tipo resultante de expressões compostas. Ou seja, as regras contidas pela linguagem que dizia exemplo: (int + float -> float). O respectivo método que faz isso é chamado de “resolveResultType();”
- Função “typesCompatible()”; foi criada para validar atribuições, gerando erro com número da linha em caso de tipos incompatíveis.

Modificado

- As funções de análise de expressão no geral foram modificadas, para assim conseguir retornar o tipo resultante da operação e garantir que os operandos são compatíveis. Sendo assim, funções modificadas:
 - procDecl() e procIdentList(): Adicionam variáveis na tabela de Símbolos.
 - procAssignStmt(): valida o tipo da expressão com o tipo da variável.

- `procExpression()`: garante que operadores relacionais têm operandos compatíveis.
- `procFactor()`: obtém o tipo de variáveis.
- `procCondition()`, `procWhileStmt()`, `procIfStmt()`, `procRepeatStmt()`: validam se a condição é do tipo inteiro.
- `procReadStmt()` e `procWriteStmt()`: verificam se variáveis usadas estão declaradas.

Essas modificações foram feitas para retornar conforme disse, tipos, pois a análise semântica depende da propagação de tipos ou erros através das regras da gramática, por isso as alterações maiores foram feitas no analisador sintático.

Apresentação dos Testes:

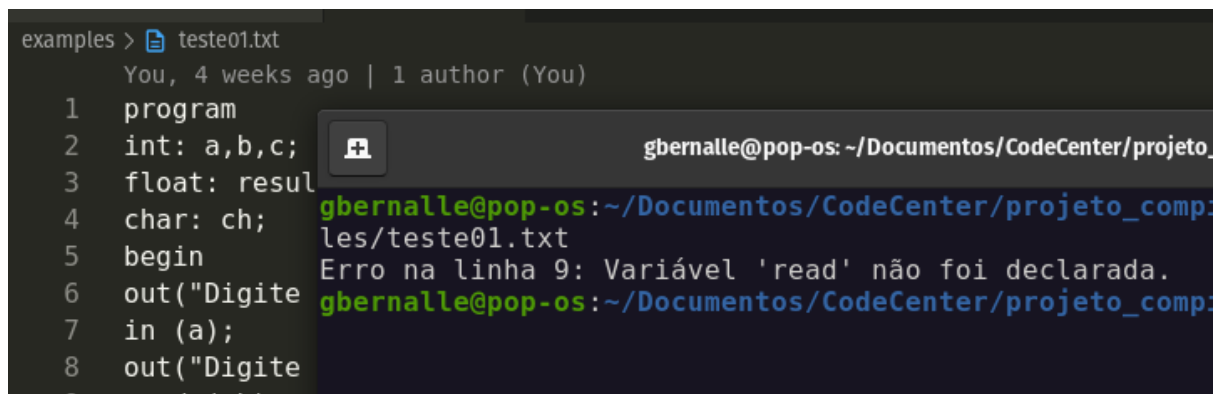
Para execução dos casos, basta digitar no terminal:

```
javac .\Main.java | java Main '.\examples\[teste0X].txt' - Windows
javac ./Main.java | java Main './examples/[teste0X].txt' - Linux
```

Substituindo **[teste0X]** pelo teste que deseja analisar

Não alterou, visto que a “main” não sofreu alterações.

Teste 01:



```
examples > teste01.txt
You, 4 weeks ago | 1 author (You)
1 program
2 int: a,b,c;
3 float: read;
4 char: ch;
5 begin
6 out("Digite
7 in (a);
8 out("Digite
9 read (ch);

gbernalles@pop-os: ~/Documentos/CodeCenter/projeto_compilacao/les/teste01.txt
Erro na linha 9: Variável 'read' não foi declarada.
```

Caso Teste - 01 (Testes, Roteiro Trabalho 2025.1)

A palavra-chave `read` não existe na linguagem, e o analisador corretamente achou que era um identificador.

Teste 02:

```
examples > teste02.txt
You, 4 weeks ago | 1 author (You)
1 program
2 float: raio, area$ = 0.0;
3 begin
4 repeat
5 in(raio);
6 char: resposta;
7 if (raio > 0.0) then
8 area = 3. * raio * raio;
9 out (area);

gbernalles@pop-os: ~/Documentos/CodeCenter/projetos/teste02.txt
02: Lexema nao esperado [=]
```

Caso Teste - 02 (Testes, Roteiro Trabalho 2025.1)

Identificador `area$` é inválido, visto que `$` não está na definição da gramática (identificadores só permitem letras, dígitos e `_`). E atribuição na declaração não é suportada pela gramática.

Teste 03:

```
examples > teste03.txt
You, 4 weeks ago | 1 author (You)
1 program
2 int: a, b, aux;
3 begin
4 in (a);
5 in(b);
6 if (a>b) then
7 int aux;
8 aux = b;
9 b = a;

gbernalles@pop-os: ~/Documentos/CodeCenter/projetos/teste03.txt
07: Lexema nao esperado [aux]
```

Caso Teste - 03 (Testes, Roteiro Trabalho 2025.1)

A sintaxe correta é igual a da linha 2 - `int: aux;` e não igual mostra na linha 7 - `int aux;`

Teste 04:

```
examples > teste04.txt
You, 4 weeks ago | 1 author (You)
1 program
2 a, b, c, maior, outro: int;
3 begin
4 repeat
5 out("A");
6 in(a);
7 out("B");
8 in(b);
9 out("C");

gbernalles@pop-os: ~/Documentos/CodeCenter/projetos/teste04.txt
02: Lexema nao esperado [a]
```

Caso Teste - 04 (Testes, Roteiro Trabalho 2025.1)

O código está ao contrário, mostrando o tipo por último e depois as atribuições de variáveis, logo está errado mesmo.

Teste 05:

```
examples > teste05.txt
You, 4 weeks ago | 1 author (You)
1 programa
2 declare
3 inteiro: pontuacao, pontuacaoM
4 char: pontuacaoMinima
5 begin
6 disponibilidade = 'S';
7 pontuacaoMinima = 50;
8 pontuacaoMaxima = 100;
9 out("Pontuacao Candidato: ");
```

gbernalle@pop-os: ~/Documentos/CodeCent
les/teste05.txt
01: Lexema nao esperado [programa]

Caso Teste - 05 (Testes, Roteiro Trabalho 2025.1)

Escreveu programa em vez de program. Usou inteiro em vez de int. Por fim, tem declare, que não é reconhecido pela linguagem.

Teste 06:

```
examples > teste06.txt
You, 4 hours ago | 1 author (You)
1 program
2 int : x, y, z;
3 float : media;
4 char : letra;
5
6 begin
7     x = 10;
8     y = 20;
9     z = x + y;
```

gbernalle@pop-os: ~/Documentos/CodeCent
les/teste06.txt

Caso Teste - 06 (Testes, Roteiro Trabalho 2025.1)

Conforme esperado, não gera erros.

OBS: Professora, por favor não esqueça meu ponto extra !!!